Lecture 11: Concurrency Part I

Professor G. Sandoval

Some slides derived from : Junfeng Yang and Tanenbaum/Bo,

Outline

- Critical section requirements
- Implementing locks
- Readers-writer lock

Recall: Banking example

```
int balance = 0;
int main()
{
        pthread_t t1, t2;
        pthread_create(&t1, NULL, deposit, (void*)1);
        pthread_create(&t2, NULL, withdraw, (void*)2);
        pthread_join(t1, NULL);
        pthread_join(t2, NULL);
        printf("all done: balance = %d\n", balance);
        return 0;
}
```

```
void* deposit(void *arg)
{
        int i;
        for(i=0; i<1e7; ++i)
                ++ balance;
}
```

```
void* withdraw(void *arg)
{
        int i;
        for(i=0; i<1e7; ++i)
                -- balance;
}
```

Results of the banking example

```
$ gcc –Wall –lpthread –o bank bank.c
$ bank
all done: balance = 0
$ bank
all done: balance = 140020
$ bank
all done: balance = -94304
$ bank
all done: balance = -191009
```

Why?

A closer look

```
$ objdump –d bank
…
08048464 <deposit>:
…                                   // ++ balance
8048473:  a1 80 97 04 08            mov    0x8049780,%eax
8048478:  83 c0 01                  add    $0x1,%eax
804847b:  a3 80 97 04 08            mov    %eax,0x8049780
…

0804849b <withdraw>:
…                                   // -- balance
80484aa:  a1 80 97 04 08            mov    0x8049780,%eax
80484af:  83 e8 01                  sub    $0x1,%eax
80484b2: a3 80 97 04 08             mov    %eax,0x8049780
…
```

One possible schedule

CPU 0		CPU 1
	balance: 0	
mov 0x8049780,%eax		
eax0: 0		
add $0x1,%eax		
eax0: 1		
mov %eax,0x8049780		
balance: 1		
		mov 0x8049780,%eax
	eax1: 1	
		sub $0x1,%eax
	eax1: 0	
		mov %eax,0x8049780
	balance: 0	

time

One deposit and one withdraw, balance unchanged. Correct

Another possible schedule

CPU 0		CPU 1
	balance: 0	
mov 0x8049780,%eax		
	eax0: 0	
add $0x1,%eax		
	eax0: 1	
		mov 0x8049780,%eax
	eax1: 0	
mov %eax,0x8049780		
	balance: 1	
		sub $0x1,%eax
	eax1: -1	
		mov %eax,0x8049780
	balance: -1	

time

One deposit and one withdraw,
balance becomes less. Wrong!

Race Conditions

- If two processes or threads need to update some data at the same time, we may have a *race condition*
- These problems are difficult to debug – problem only occurs when the timing is just right and is hard to reproduce
- Note that we don't actually need multiple processors here for a mistake to occur!
- Just *preemption* at the wrong time

Mutual Exclusion

- The key trouble we ran into was that the **read-update-write sequence** on a shared resource could be interleaved between two processes

  ($read_1$-$read_2$-$update_2$-$write_2$-$update_1$-$write_1$)

- The way to solve this is through ***mutual exclusion*** –making sure that **only one process** has access to the shared resource at a time

Critical Regions/Sections

- We can formulate the problem by classifying what programs do into two parts
  1. The majority of the time they do things that **don't require synchronization**; the things they do only affect non-shared resources
  2. Some of the time, they need to access shared memory or files; we call this a ***critical region*** **or** ***critical section*** of the program
- If we can arrange it so that two programs are **never in a critical section at the same time**, we can avoid race conditions

Avoid race conditions

- ❑ **Critical section**: a segment of code that accesses a shared variable (or resource)

- ❑ No more than one thread in critical section at a time.

```
// ++ balance
mov    0x8049780,%eax
add    $0x1,%eax
mov    %eax,0x8049780
...

// -- balance
mov    0x8049780,%eax
sub    $0x1,%eax
mov    %eax,0x8049780
...
```

Critical Section Requirements

- Safety (aka mutual exclusion): no more than one thread in critical section at a time.

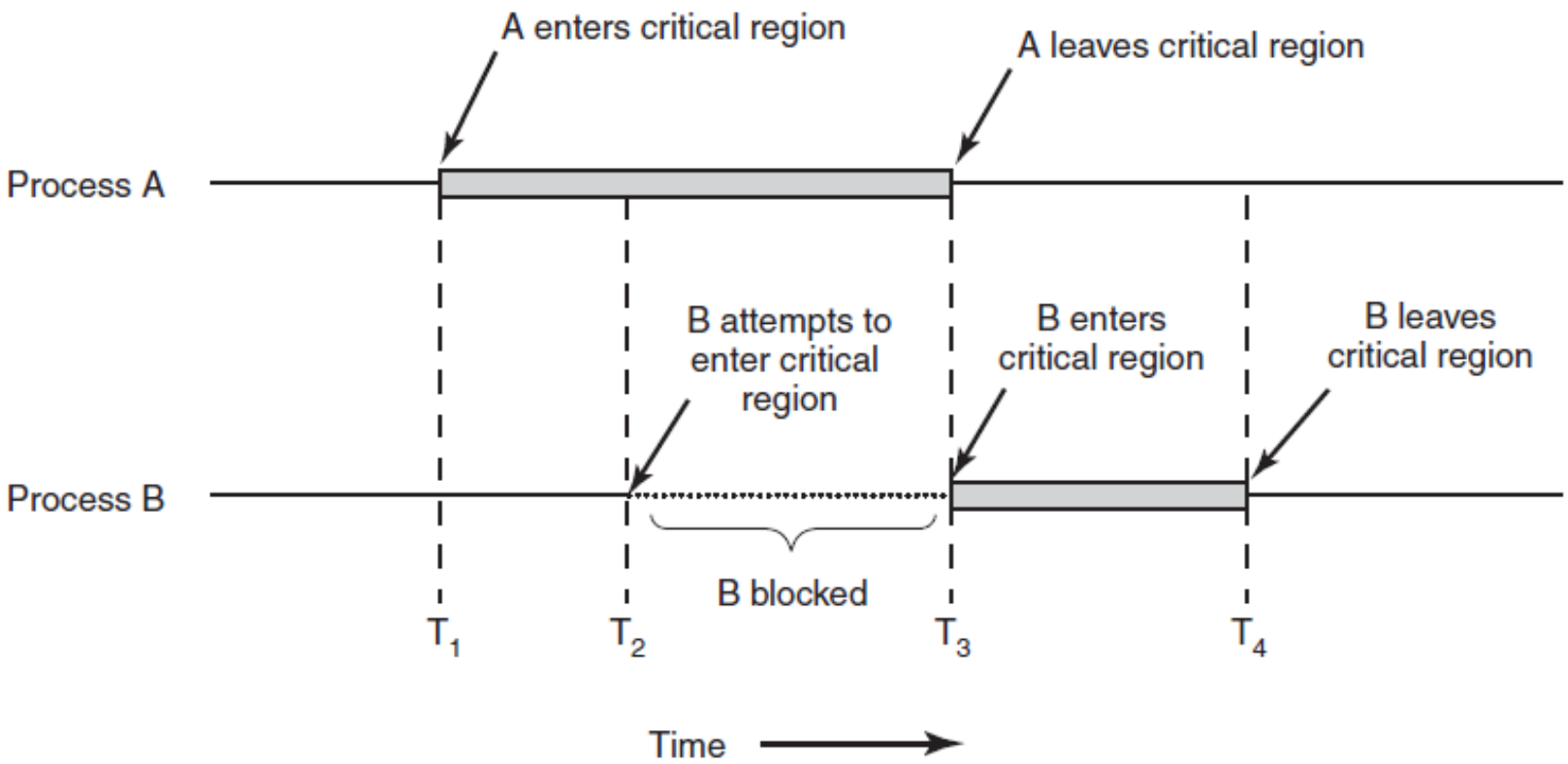
- Liveness (aka progress):
  - If multiple threads simultaneously request to enter critical section, must allow one to proceed
  - Must not depend on threads outside critical section

- Bounded waiting (aka starvation-free)
  - Must eventually allow waiting thread to proceed

- Makes no assumptions about the speed and number of CPU
  - However, assumes each thread makes progress

CS desirable properties

- **Efficient**: don't consume too much resource while waiting
  - Don't busy wait (spin wait). Better to relinquish CPU
    - and let other thread run
- **Fair**: don't make one thread wait longer than others. Hard to do efficiently
- **Simple**: should be easy to use


A enters critical region
A leaves critical region
Process A
B attempts to enter critical region
B enters critical region
B leaves critical region
Process B
B blocked
T1
T2
T3
T4
Time

Implementing critical section using locks

- **lock(l):** acquire lock exclusively; wait if not available
- **unlock(l):** release exclusive access to lock

```
void* deposit(void *arg)
{
        int i;

        for(i=0; i<1e7; ++i) {
                pthread_mutex_lock(&l);
                ++ balance;
                pthread_mutex_unlock(&l);
        }
}
```

```
void* withdraw(void *arg)
{
        int i;

        for(i=0; i<1e7; ++i) {
                pthread_mutex_lock(&l);
                -- balance;
                pthread_mutex_unlock(&l);
        }
}
```

Outline

- Critical section requirements
- Implementing locks
- Readers-writer lock

Implementing locks: version 1

- **Can cheat on uniprocessor**: implement locks by disabling and enabling **interrupts**

```
lock()
{
        disable_interrupt();
}
```

```
unlock()
{
        enable_interrupt();
}
```

- **Good**: simple!
- **Bad**:
  - Both operations are **privileged**, can't let user program use
  - Doesn't work on multiprocessors

Implementing locks: version 2

- **Peterson's algorithm**: software-based lock implementation

- **Good**: doesn't require much from hardware
  - Only assumptions:
    - Loads and stores are **atomic**
    - They execute **in order**
  - Does not require special hardware instructions
- **Bad**:
  - Hard to get right

Peterson's Algorithm

- Each process indicates its *interest* by setting its entry in the "interested" array
- Then, it sets the global *turn* variable to its own process number
- Finally, loop until *turn* indicates that it's our turn *and* we see that the other process is no longer interested
- There is still a race – but regardless of the winner, only one process will get to enter its critical region

Peterson's Algorithm

```
#define FALSE 0
#define TRUE 1
#define N 2                              /* number of processes */

int turn;                                /* whose turn is it? */
int interested[N];                       /* all values initially 0 (FALSE) */

void enter_region(int process);          /* process is 0 or 1 */
{
  int other;                             /* number of the other process */
  other = 1 – process;                   /* the opposite of process */
  interested[process] = TRUE;            /* show that you are interested */
  turn = process;                        /* set flag */
  while (turn == process && interested[other] == TRUE) {
    // Do nothing
  }
}

void leave_region(int process) {         /* process: who is leaving */
  interested[process] = FALSE;           /* indicate departure from
                                            critical region */
}
```

Software-based lock: 1st attempt

```
// 0: lock is available, 1: lock is held by a thread
int flag = 0;

lock()
{
        while (flag == 1)
            ; // spin wait
        flag = 1;
}

unlock()
{
        flag = 0;
}
```

- **Idea**: use one flag, test then set; if unavailable, **spin-wait**
- **Problem**?
  - Not safe: both threads can be in critical section
  - Not efficient: busy wait, particularly bad on uniprocessor (will solve this later)

When poll is active, respond at **pollev.com/gsandoval**

Race Conditions are easy !!

True

False

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Bug in software lock, 1st attempt

```
lock()
{
        1: while (flag == 1)
             ; // spin wait
        2: flag = 1;
}
```

```
unlock()
{
        3: flag = 0;
}
```

Software-based lock

- 2nd attempt: use per thread flags, set then test, to achieve mutual exclusion
  - Not live: can deadlock
- 3rd attempt: strict alternation to achieve mutual exclusion
  - Not live: depends on threads outside critical section
- Final attempt: combine above ideas
- Problem
  - It's hard!
  - N>2 threads? (Lamport's Bakery algorithm)
  - Modern out of order processors?

Implementing locks: version 3

```
// 0: lock is available, 1: lock is held by a thread
int flag = 0;
```

```
lock()
{
        while(test_and_set(&flag))
            ;
}
```

```
unlock()
{
        flag = 0;
}
```

- Problem with the test-then-set approach: test and set are not atomic
- Fix: special atomic operation
  - Atomically returns *lock and sets *lock to 1
    -
```
int test_and_set (int *lock) {
    int old = *lock;
    *lock = 1;
    return old;
}
```

x86 Atomic Locking

- A similar instruction exists on x86: **xchg REG, MEM**
- Atomically exchanges the contents of a register and a memory location
- You can see that this is equivalent to TSL if the register is set to 1

xv6 Lock using XCHG

```
void
acquire(struct spinlock *lk)
{
  pushcli(); // disable interrupts to avoid deadlock.
  if(holding(lk))
    panic("acquire");

  // The xchg is atomic.
  // It also serializes, so that reads after acquire are not
  // reordered before it.
  while(xchg(&lk->locked, 1) != 0)
    ;

  // Record info about lock acquisition for debugging.
  lk->cpu = cpu;
  getcallerpcs(&lk, lk->pcs);
}
```

- xchg reg, addr: atomically swaps *addr and reg
- Most spin locks on x86 are implemented using this instruction
  - xv6 spinlock.h, spinlock.c, x86.h

Spin-wait or block?

- Problem: waste CPU cycles
  - Worst case: prev thread holding a busy-wait lock gets preempted, other threads try to acquire the same lock
- On uniprocessor: should not use spin-lock
  - Yield CPU when lock not available (need OS support)
- On multi-processor
  - Thread holding lock gets preempted ➔ ???
  - Correct action depends on how long before lock release
    - Lock released "quickly" ➔ ?
    - Lock released "slowly" ➔ ?

Mutexes

- A **mutex** is a way to have mutual exclusion **without busy waiting**

- Implementation is very **similar** to the busy-wait version of critical regions, but instead of looping, **we yield the CPU**

Mutex Code

```
mutex_lock:
        TSL REGISTER,MUTEX      | copy mutex to register and set mutex to 1
        CMP REGISTER,#0         | was mutex zero?
        JZE ok                  | if it was zero, mutex was unlocked, so return
        CALL thread_yield       | mutex is busy; schedule another thread
        JMP mutex_lock          | try again
ok:     RET                     | return to caller; critical region entered

mutex_unlock:
        MOVE MUTEX,#0           | store a 0 in mutex
        RET                     | return to caller
```

Critical Section

```
enter_region:
        TSL REGISTER,LOCK          | copy lock to register and set lock to 1
        CMP REGISTER,#0            | was lock zero?
        JNE enter_region           | if it was not zero, lock was set, so loop
        RET                        | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0               | store a 0 in lock
        RET                        | return to caller
```

Mutex Code

```
mutex_lock:
        TSL REGISTER,MUTEX         | copy mutex to register and set mutex to 1
        CMP REGISTER,#0            | was mutex zero?
        JZE ok                     | if it was zero, mutex was unlocked, so return
        CALL thread_yield          | mutex is busy; schedule another thread
        JMP mutex_lock             | try again
ok:     RET                        | return to caller; critical region entered

mutex_unlock:
        MOVE MUTEX,#0              | store a 0 in mutex
        RET                        | return to caller
```

Problem with simple yield

```
lock()
{
        while(test_and_set(&flag))
            yield();
}
```

- Problem:
  - Still a lot of context switches: thundering herd
  - Starvation possible
- Why? No control over who gets the lock next
- Need explicit control over who gets the lock

Implementing locks: version 4

```
lock() {
  while (test_and_set(&flag)))
    add myself to wait queue
    yield
  ...
}
```

```
unlock() {
    flag = 0
    if(any thread in wait queue)
      wake up one wait thread
  ...
}
```

Lock from another thread?

- The idea: add thread to queue when lock unavailable; in unlock(), wake up one thread in queue
- Problem I: lost wakeup
  - Fix: use a spin_lock or lock w/ simple yield!
  - Doesn't avoid spin-wait, but make wait time short
- Problem II: wrong thread gets lock
  - Fix: unlock() directly transfers lock to waiting thread

Lost wakeup

```
lock() {
  1: while (test_and_set(&flag)))
      2: add myself to wait queue
      3: yield
  …
}
```

```
unlock() {
    4: flag = 0
    5: if(any thread in wait queue)
       6: wake up one wait thread
  …
}
```

Implementing locks: version 4, the code

```
typedef struct __mutex_t {
    int flag;        // 0: mutex is available,  1: mutex is not available
    int guard;       // guard lock to avoid losing wakeups
    queue_t *q;      // queue of waiting threads
} mutex_t;
```

```
void lock(mutex_t *m) {
    while (test_and_set(m->guard))
        ; //acquire guard lock by spinning
    if (m->flag == 0) {
        m->flag = 1; // acquire mutex
        m->guard = 0;
    } else {
        enqueue(m->q, self);
        m->guard = 0;
        yield();
    }
}
```

```
void unlock(mutex_t *m) {
    while (test_and_set(m->guard))
        ;
    if (queue_empty(m->q))
        // release mutex; no one wants mutex
        m->flag = 0;
    else
        // direct transfer mutex to next thread
        wakeup(dequeue(m->q));
    m->guard = 0;
}
```

Outline

- Critical section requirements
- Implementing locks
- Readers-writer lock

Readers-Writers problem

- A reader is a thread that needs to look at the shared data but won't change it
- A writer is a thread that modifies the shared data
- Example: making an airline reservation
- Courtois et al 1971

Solving Readers-Writers w/ regular lock

```
lock_t lock;
```

Writer

```
lock (&lock);
. . .
// write shared data
. . .
unlock (&lock);
```

Reader

```
lock (&lock);
. . .
// read shared data
. . .
unlock (&lock);
```

- **Problem**: unnecessary synchronization
  - Only one writer can be active at a time
  - However, any number of readers can be active simultaneously!
- **Solution**: acquire lock for read mode and write mode

Readers-writer lock

```
rwlock_t lock;
```

Writer

```
write_lock (&lock);
. . .
// write shared data
. . .
write_unlock (&lock);
```

Reader

```
read_lock (&lock);
. . .
// read shared data
. . .
read_unlock (&lock);
```

- read_lock: acquires lock in read (shared) mode
  - If lock is not acquired or in read mode ➔ success
  - Otherwise, lock is in write mode ➔ wait
- write_lock: acquires lock in write (exclusive) mode
  - If lock is not acquire ➔ success
  - Otherwise ➔ wait

Implementing readers-writer lock

```
struct rwlock_t {
    int nreader;      // init to 0
    lock_t guard;     //  init to unlocked
    lock_t lock;      // init to unlocked
};

write_lock(rwlock_t *l)
{
    lock(&l->lock);
}

write_unlock(rwlock_t *l)
{
    unlock(&l->lock);
}
```

```
read_lock(rwlock_t *l)
{
    lock(&l->guard);
    ++ nreader;
    if(nreader == 1) // first reader
        lock(&l->lock);
    unlock(&l->guard);
}

read_unlock(rwlock_t *l)
{
    lock(&l->guard);
    -- nreader;
    if(nreader == 0) // last reader
       unlock(&l->lock);
    unlock(&l->guard);
}
```

Problem: may starve writer!

Backup slides

Software-based locks: 2nd attempt

```
// 1: a thread wants to enter critical section, 0: it doesn't
int flag[2] = {0, 0};

lock()
{
        flag[self] = 1; // I need
        lock while (flag[1- self] ==
        1)
}          ; // spin wait

unlock()
{
        // not any more
        flag[self] = 0;
}
```

- Idea: use per thread flags, set then test, to achieve mutual exclusion
- Why doesn't work?
  - Not live: can deadlock

Software-based locks: 3rd attempt

```
// whose turn is it?
int turn = 0;

lock()
{
        // wait for my turn
        while (turn == 1 –
        self)
}           ; // spin wait
```

```
unlock()
{
        // I'm done. your turn
        turn = 1 – self;
}
```

- Idea: strict alternation to achieve mutual exclusion
- Why doesn't work?
  - Not live: depends on threads outside critical section

Software-based locks: final attempt (Peterson's algorithm)

```
// whose turn is it?
int turn = 0;
// 1: a thread wants to enter critical section, 0: it doesn't
int flag[2] = {0, 0};
```

```
lock()
{
        flag[self] = 1; // I need
        lock turn = 1 – self;
        // wait for my turn
        while (flag[1-self] == 1
            && turn == 1 – self)
            ;  // spin wait while the
               // other thread has intent
               // AND it is the other
               // thread's turn
}
```

```
unlock()
{
        // not any more
        flag[self] = 0;
}
```

- Why works?
  - Safe?
  - Live?
  - Bounded wait?